

UNIVERSITÉ DE FRANCHE-COMTÉ

EXAMENS C.T.U.

SESSION DE : Mai 2026

Centre d'examen de : Le Lamentin - 2

Diplôme préparé : Master IZA

U.E. et / ou épreuve : SPP

Date : 21/05/2026

Nombre de copies utilisées : 4

Nom patronymique : Zanon
 Nom marital :
 Prénoms : Julie
 Né(e) le : 2004
 N° d'étudiant :
 Numéro de place : 49

NOTE
de 0 à 20

01,5 / 20

APPRÉCIATIONS EXPLIQUANT LA NOTE CHIFFRÉE :

Quasiment aucun acquis

Ne pas écrire
dans cette marge

Exercice 1:

1) C'est seulement si un étudiant travaille qu'il a de bonnes notes:

$T \rightarrow N$

f

2) Un étudiant n'a de bonnes notes que s'il travaille : $T \vee (T \wedge N)$

f

3) Pour un étudiant, le travail est une condition nécessaire à l'obtention de bonnes notes:

~~$T \rightarrow N$~~
 $T \rightarrow \neg N$

faux } choisi, pas équivalent

f

4) Un étudiant a de mauvaises notes, à moins qu'il ne travaille:

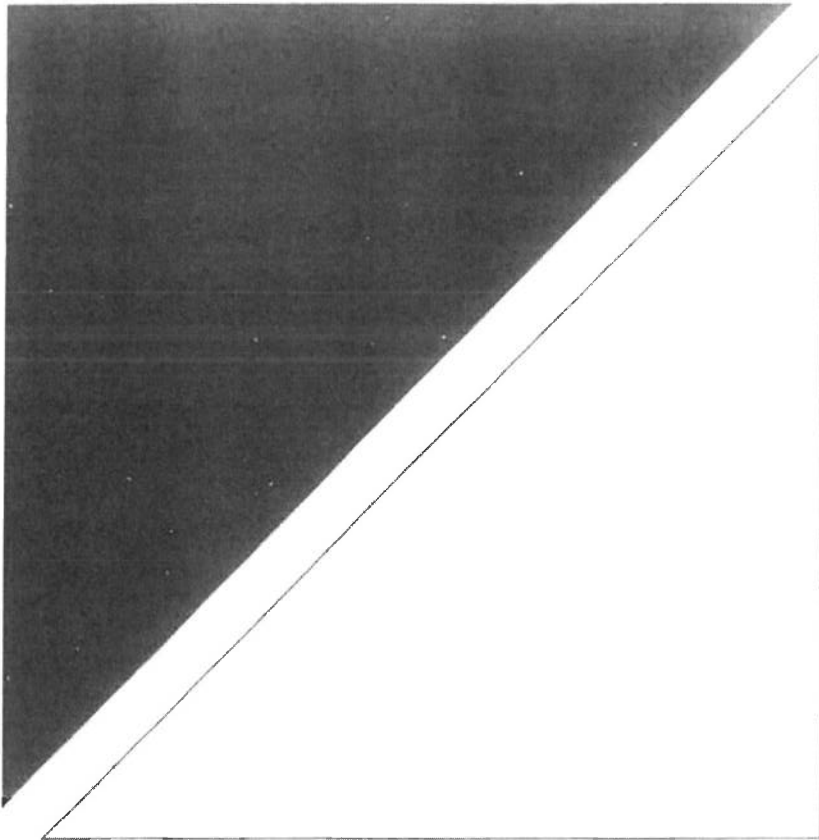
$\neg N \vee T$

J

5) Malgré son travail, un étudiant a de mauvaises notes: $T \wedge \neg N$

J

1



6. Un étudiant travaille seulement s'il a des bonnes notes :

$$N \rightarrow T \vee (\neg N \wedge \neg T)$$

F

7. À quoi bon travailler, si c'est pour avoir de mauvaises notes?

Cette traduction est impossible. Pour formaliser il faut des affirmations avec une relation de cause à effet.

T

0,5

8. Un étudiant a de bonnes notes sauf s'il ne travaille pas :

$$\neg N \wedge \neg T$$

$$\neg T \Rightarrow \neg N$$

F

Exercice 2 :

Soit P1 "a implique b" et P2 "Soit non a, soit a et b" traduite par :

$$P1 : a \rightarrow b$$

$$P2 : \text{not } a \vee (a \wedge b) \text{ donc si } a \text{ on a } a \wedge b$$

let P1 (a: int, b: int): int
if a ↗ erreur de type
then b
done

let P2 (a: int, b: int): int
if not a then break
else if a
then a $\wedge$ b

Ø

Exercice 3

- 1) type t
~~predicate p x y~~ / *non connue*
~~predicate p y z~~
~~predicate transitif = exist x. p x z~~
- 2) type t
~~predicate p t t~~
~~predicate asym = not(exist x. p y x)~~ *incomplet*
- 3) lemme est asym : ~~bratt predicate irrefl and predicate transitif~~
~~exist predicate asym~~

φ

φ

φ

Exercice 4

- 1) ~~length(Nil) = Nil = 0~~
~~length(Cons(a, l)) = Cons'a list'a → Cons'b(list'b)~~

f

f

φ

comme pas connue

2) ~~X~~

- 3) let rec function len (l: list 'a): int
~~= match l with~~
~~| Nil → 0~~
~~| Cons a → Cons l~~
~~end~~

φ

- 4) lemme l : ~~exist (length Nil = 0)~~

φ

- 5) lemme l2 : ~~0 < len(l)~~ *+ forall*

φ

- 6) Il faut spécifier que la longueur de la liste est strictement
non nulle et supérieure à 0.
non

φ

Exercice 5

- 1) Soit le Triplet de Hoare tel que $\{P\} C \{Q\}$ avec $\{P\}$ la precondition, C le programme et $\{Q\}$ la postcondition

$$\{P\} = \{0 < \text{length } t\}$$

$$\{Q\} = \{ \text{length } t - 1 < \text{imax} \leq \text{length } t \}$$

faux

$\emptyset$

alors dans ce cas on aura $\text{length } t - 1 < \text{imax} \leq \text{length } t$

- 2) ici pour la postcondition on aura

$$\text{ensures } [t[\text{imax}]]$$

n'est pas une formule

$\emptyset$

pour être sûr que le imax tracer fait pointer effectivement de la liste

- 3) le programme avance au fur et à mesure avec k qui diminue de 1 à chaque fois c'est l'invariant de boucle, cette proposition est vraie à tout moment

$$\text{invariant : } k \geq k - 1$$

$\emptyset$

et le programme s'arrête que $0 \leq k$ n'est plus correct
alors que $k = 0$

$$\text{variant : } k \leq 0$$